

[← Go Back](#)

Hardware

MKR WAN 1310



Tutorials

LoRa® LED Control with MKR WAN 1310

LoRa®-Based Messaging with MKR WAN 1310

Send Data Using LoRa® Technology with MKR WAN 1310

LoRa® Sensor Data with MKR WAN 1310

LoRa® Farming with MKR WAN 1310

LoRa® Regional Parameters in Arduino® MKR WAN 1310

MKR WAN Examples

Connecting MKR WAN 1310 to The Things Network (TTN)

MKR WAN 1310 with MKR GPS Shield

Home / Hardware / MKR WAN 1310 / **LoRa® LED Control with MKR WAN 1310**

LoRa® LED Control with MKR WAN 1310

Learn how to connect two boards using LoRa® technology and control an LED remotely from another board.

Author · Karl Söderby · Last revision · 02/28/2025

In this tutorial, we will set up two MKR WAN 1310's to host a remote LED control. One board will be set up as a sender with a pushbutton that when it is pressed, an LED on the other board will turn on or off.

We will use the **LoRa** library for the communication, and we will not use any external service.

Hardware & Software Needed

Arduino IDE ([online](#) or [offline](#))

LoRa library installed, see the [github repository](#)

2x Arduino MKR WAN 1310 ([link to store](#))

2x antenna ([link to store](#))

1x Pushbutton

1x Generic LED

Jumper wires

Breadboard(s)

2x Micro USB cable

ON THIS PAGE

Hardware & Software Needed

Circuit

Sender Circuit

Receiver Circuit

Let's Start

Creating the Program +

Complete Code +

Upload Sketch and Testing the Program +

Experimenting with This Setup

Conclusion

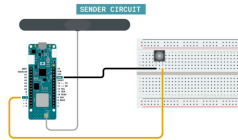
Trademark Acknowledgments

[Help](#)

Circuit

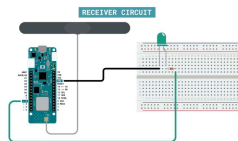
Follow the wiring diagrams below to create the circuits for the sender and receiver boards.

Sender Circuit



The sender circuit.

Receiver Circuit



The receiver circuit.

Let's Start

In this tutorial, we will achieve a basic trigger using the LoRa® technology. When we press a button on one of the MKR WAN 1310 boards, an LED will light up on another. This is quite a simple setup, but it can be useful for long range, low power communication!

But let's take a look at what we need to include in the code. As we are using two different boards, we will also need to create two separate sketches.

To create the sender sketch, we

Initialize the **SPI** and **LoRa** libraries.

Create a counter variable.

Set the radio frequency to 868E6 (Europe) or 915E6 (North America).

Create an if statement that checks if the button is pressed.

If button is pressed, begin a packet, and print the text "button pressed".

End packet.

Increase counter each loop and print it in Serial Monitor.

To create the receiver sketch, we will have to do the following steps:

Initialize the **SPI** and **LoRa** libraries.

Create a string with the message "button pressed" stored.

Set the radio frequency to 868E6 (Europe) or 915E6 (North America).

Create a function to parse incoming packet.

Check if incoming string matches string we have created.

Each time string matches, switch a boolean.

Use boolean as a trigger to turn an LED on or off.

Creating the Program

1. First, let's make sure we have the drivers installed. If we are using the Cloud Editor, we do not need to install anything. If we are

done by navigating to **Tools > Board > Board Manager...** Here we need to look for the **Arduino SAMD boards (32-bits Arm® Cortex®-M0+)** and install it.

2. Now we need to download the **LoRa** library from [this repository](#), where you can install it by navigating to **Sketch > Include Library > Add .ZIP Library...** in the offline IDE.

Programming the Sender

In the initialization we will include the **SPI** and **LoRa** libraries. We will then create the `counter` variable to track how many times we have pressed the button. Next, we will create the `button` and `buttonState` variables, used to assign the pushbutton to pin 2, and to read the state of it.

```
1 #include <SPI.h>
2 #include <LoRa.h>
3
4 int counter = 0;
5 int button = 2;
6 int buttonState;
```

In the `setup()` we will first define the `button` pin as an `INPUT_PULLUP`. We will then begin serial communication, where we will use the command `while(!Serial);` to prevent the program from running until we open the Serial Monitor.

We will then initialize the **LoRa** library, where we will set the radio

communication. If we are located in North America, we need to change this to 915E6.

```
1 void setup() {  
2   pinMode(button, INPUT);  
3  
4   Serial.begin(9600);  
5  
6   while (!Serial);  
7   Serial.println("LoRa  
8  
9   if (!LoRa.begin(868E6))  
10    Serial.println("Starting  
11    while (1);  
12  }  
13  
14  delay(1000);  
15 }
```

In the `loop()` we start by reading the button, and store the state in the `buttonState` variable.

If the button is pressed, we begin a packet by using the command, `LoRa.beginPacket()`. We then print the message "button pressed", which is done by using the `LoRa.print()` function. We then broadcast the package, using the `LoRa.endPacket()` command.

```
1 void loop() {
2   buttonState = digitalRead(buttonPin);
3
4   if (buttonState == LOW) {
5     // send packet
6     LoRa.beginPacket();
7     LoRa.print("button pressed");
8     LoRa.endPacket();
9     counter++;
10    Serial.print("Send packet: ");
11    Serial.println(counter);
12    delay(500);
13  }
14 }
```

Programming the Receiver

In the initialization we will first include the **SPI** and **LoRa** libraries. Then we will create two strings: one empty, and one with the message "button pressed" stored. The `contents` string will be used to store incoming data, and the `buttonPress` string will be used to compare the contents with the incoming data.

We then create the boolean `x`, which will switch from true to false each time `buttonPress` matches `contents`. We will also assign the `led` variable to pin 2.

```
1 #include <SPI.h>
2 #include <LoRa.h>
3
4 String contents = "";
5 String buttonPress = "button pressed";
6 bool x;
7
8 int led = 2;
```

In the `setup()` we will first define the `led` pin as an output. We will then begin serial communication, where we will use the command `while(!Serial);` to prevent the program from running until we open the Serial Monitor.

We will then initialize the **LoRa** library, where we will set the radio frequency to 868E6, which is used in Europe for LoRa® communication. If we are located in North America, we need to change this to 915E6.

```
1 void setup() {  
2  
3   pinMode(led, OUTPUT)  
4   Serial.begin(9600);  
5   while (!Serial);  
6   //Wire.begin();  
7   Serial.println("LoRa  
8  
9   if (!LoRa.begin(868E6  
10    Serial.println("St  
11    while (1);  
12 }  
13 }
```

Inside the `loop()`, we will not be creating any packets. Instead, we will listen to incoming ones. This is done by first using the command

```
int packetSize =  
LoRa.parsePacket();
```

, and then checking for an incoming packet. If a packet comes in, it is then stored in the `contents` string. After that, we also check the RSSI (Received Signal Strength Indication), and print in the Serial Monitor.

We then compare the `contents` string to the `buttonPress` string. If it matches, we switch the `x` variable by using the command `x = !x;`. A conditional then checks whether `x` is `true` or `false` and turns on or off the LED attached to pin 2.

Finally, after the action has happened, we reset the `contents` string, otherwise, we would just pile up the messages inside the string.

```
1 void loop() {
2   // try to parse pac
3   int packetSize = Lo
4   if (packetSize) {
5     // received a pac
6     Serial.print("Rec
7
8     // read packet
9     while (LoRa.avail
10      contents += (ch
11   }
12
13   Serial.println(co
14   // print RSSI of
15   Serial.print("' w
16   Serial.println(Lo
17
18   if(contents.equal
19     x = !x;
20   }
21
22   if(x == true) {
23     digitalWrite(le
24     Serial.println(
25   }
26   else {
27     digitalWrite(le
28     Serial.println(
```

Complete Code

If you choose to skip the code building section, the complete code can be found below:

Sender Code

```
1  #include <SPI.h>
2  #include <LoRa.h>
3
4  int counter = 0;
5  int button = 2;
6  int buttonState;
7
8  void setup() {
9      pinMode(button, INP
10
11      Serial.begin(9600);
12
13      while (!Serial);
14      Serial.println("LoR
15
16      if (!LoRa.begin(868
17          Serial.println("S
18          while (1);
19      }
20      delay(1000);
21  }
22
23  void loop() {
24      buttonState = digit
25
26      if (buttonState ==
27          // send packet
28      LoRa.beginPacket(
```

Receiver Code

```
1  #include <SPI.h>
2  #include <LoRa.h>
3
4  String contents = "";
5  String buttonPress =
6  bool x;
7
8  int led = 2;
9
10 void setup() {
11
12     pinMode(led, OUTPUT
13     Serial.begin(9600);
14     while (!Serial);
15     //Wire.begin();
16     Serial.println("LoR
17
18     if (!LoRa.begin(868
19         Serial.println("S
20         while (1);
21     }
22 }
23
24 void loop() {
25     // try to parse pac
26     int packetSize = Lo
27     if (packetSize) {
28         // received a pac
```

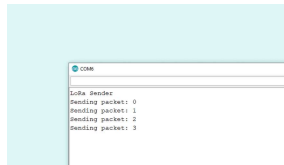
Upload Sketch and Testing the Program

Once we are finished with the coding, we can upload the sketches to each board. The easiest way to go forward is to have two separate computers, as we will need to have the Serial Monitor open for both boards. Alternatively, we can use a Serial interfacing program called Putty. But for demonstration purposes, it is good to use two computers. This way you can move the

boards further away from each other while testing the signal.

Sending Packet

After we have uploaded the code to the **sender**, we need to open the Serial Monitor to initialize the program. If everything is working, it will start listening for button presses. If the button is pressed, it will print the message "Sending packet: x" where x represents the number of times the packet has been sent.



Sending packets.

Receiving Packet

After we have uploaded the code to the **receiver**, we need to open the Serial Monitor to initialize the program. If everything works, we should now pick up any package we send from the other device. The package we receive should contain the message "button pressed", followed by RSSI (Received Signal Strength Indication). The closer this value is to 0, the stronger the signal are. After we receive the value, the LED will turn on or off, and print its state in the Serial Monitor.



Receiving packets.

Experimenting with This Setup

Now that we have communication between the boards, we can do a simple test with the signal. If we move the sender device away from the receiver device, we will start noticing changes in the RSSI. For example, while conducting this test, the sender device was moved around 20 meters away from the receiver, which decreased the RSSI to about -60.

Making Sender Device Mobile

As of now, we have a quite immobile setup, but this can be fixed by making the sender device mobile. A good way of testing this out is to make the sender device run on a battery instead. The MKR WAN 1310 has a battery connector that any battery with a JST PH connector can connect to. A standard 3.7 LiPo battery can power the MKR WAN 1310 for a longer time.

Troubleshoot

If the code is not working, there are some common issues we might need to troubleshoot:

- Antenna is not connected properly.

- The radio frequency is

for Europe and 915E6 for Australia & North America.

We have not opened the Serial Monitor.

We are using the same computer for both boards without a serial interfacing program.

Conclusion

This tutorial demonstrates a simple, yet powerful communication setup, featuring two MKR WAN 1310 boards and how to remotely control an LED over the LoRa®-based network.

Trademark

Acknowledgments

LoRa® is a registered trademark of Semtech Corporation.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

